Radac_wk3

Ronnie Bailey-Steinitz

2026-07-08

Skills Learning – Lecture

Last week, we focused on comparing groups when the response variable was quantitative and reasonably close to normal.

This week, we will ask:

> **What do we do when normality is not a good assumption?**

This happens often in ecological and biological data. Many response variables are not normally distributed.

Examples include:

- count data, such as aggression events or parasite load
- skewed data, such as hormone concentrations
- data with many zeros
- data that cannot be negative

Today we will focus on:

- recognizing when a t-test, ANOVA, or linear model may not be appropriate
- nonparametric alternatives to t-tests and ANOVA
- count data
- Poisson regression
- overdispersion
- negative binomial regression
- when log transformations may or may not help

The main question for today is:

> **What kind of analysis should I use when my response variable is not normally distributed?**

0. Load Required Packages

```
library(tidyverse)
library(janitor)
library(here)
library(rstatix) # for Dunn post-hoc test
```

1. Load and Preview the Dataset

Today we will continue using the faux gorilla hormone + behavioral dataset.

```
# load dataset
data <- readr::read_csv(here::here("Week 3/gorilla_hormone_data_week3.csv")) %>%
  janitor::clean_names()
```

```
## Rows: 240 Columns: 21
## -- Column specification --------------------------------------
## Delimiter: ","
## chr  (9): sample_id, gorilla_id, gorilla_name, group, sa...
## dbl (12): age_years, dominance_rank, group_size, fruit_a...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# preview dataset
glimpse(data)
```

```
## Rows: 240
## Columns: 21
## $ sample_id               <chr> "G001_S01", "G001_S02", "~
## $ gorilla_id              <chr> "G001", "G001", "G001", "~
## $ gorilla_name            <chr> "Gasore", "Gasore", "Gaso~
## $ group                   <chr> "BWE", "BWE", "BWE", "BWE~
## $ sample_date             <chr> "5/23/25", "8/18/25", "9/~
## $ observer                <chr> "Mark", "Sarah", "Mary", ~
## $ sex                     <chr> "F", "F", "F", "F", "F", ~
## $ age_class               <chr> "adult", "adult", "adult"~
## $ age_years               <dbl> 14.2, 14.2, 14.2, 14.2, 1~
## $ dominance_rank          <dbl> 8, 8, 8, 8, 8, 8, 8, 8, 6~
## $ group_size              <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 3~
## $ fruit_availability_index <dbl> 0.4040977, 0.3301009, 0.3~
## $ feeding_time_prop       <dbl> 0.2554745, 0.6272515, 0.5~
## $ aggression_events       <dbl> 2, 0, 1, 0, 0, 1, 1, 0, 3~
## $ parasite_load           <dbl> 46, 33, 36, 49, 39, 33, 6~
## $ respiratory_signs       <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0~
## $ injury_status           <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 1~
## $ cortisol_ng_ml          <dbl> 19.158612, 22.606257, 28.~
## $ testosterone_ng_ml      <dbl> 5.717721, 3.666248, 3.084~
## $ injury_label            <chr> "Not injured", "Not injur~
## $ play_events             <dbl> 3, 4, 7, 4, 4, 8, 5, 2, 0~
```

2. When Normality Is Not Appropriate

Many standard tests assume that the response variable, or the model residuals, are approximately normal.

This is often reasonable for continuous variables like:

- body mass
- hormone concentration

- temperature
- age

But it is often not reasonable for variables like:

- number of aggression events
- parasite load
- number of nests
- number of offspring
- number of animal sightings

Count variables have special properties:

- they are whole numbers
- they cannot be negative
- they often have many zeros
- they are often right-skewed
- the variance often increases as the mean increases

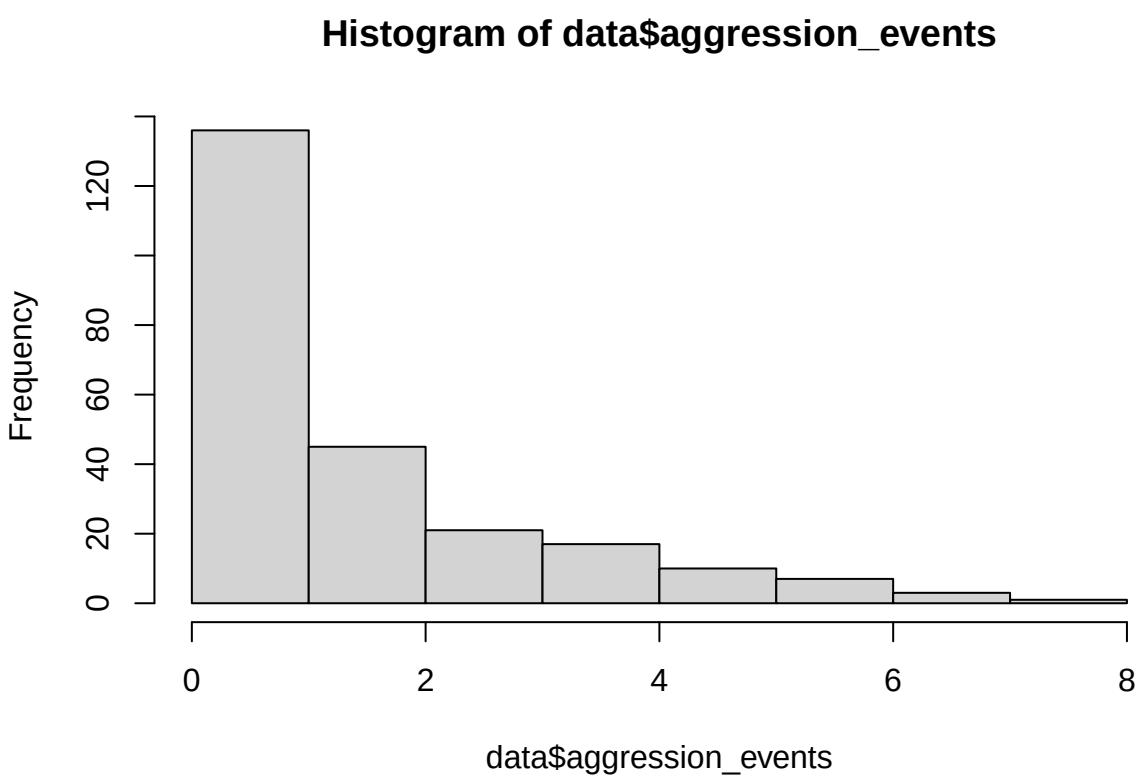
2.1 Look at count data

```
# Look at continuous and count variables
summary(data$aggression_events)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.000   0.000   1.000   1.688   2.000   8.000
```

```
# summary(data$parasite_load)

hist(data$aggression_events)
```



Key idea

A model should match the type of response variable.

```
Quantitative and approximately normal     ->  t-test, ANOVA, linear model
Non-normal group comparison               ->  Wilcoxon or Kruskal-Wallis
Count response                            ->  Poisson or negative binomial model
Binary response                           ->  logistic regression
Repeated measures                         ->  mixed effects model
```

This week we will focus on non-normal group comparisons and count responses.

3. Nonparametric Alternatives

Last week, we used t-tests and ANOVA to compare group means.
But what if the response variable is strongly skewed, has extreme outliers, or does not look approximately normal?

In those cases, we may use nonparametric tests.
Nonparametric tests are useful because they do not rely as strongly on the assumption that the response variable is normally distributed.

Today we will use:

- `wilcox.test()` as an alternative to a two-sample t-test

- `kruskal.test()` as an alternative to ANOVA

Important note:

> Nonparametric tests are not automatically better. They are used when the assumptions of the parametric test are not reasonable.

4. Wilcoxon Test: Alternative to a Two-Sample t-test

The Wilcoxon test is often used when we want to compare two independent groups, but the response variable is not approximately normal.

Example question:

> Do injured and uninjured gorillas differ in parasite load?

Response variable:

- `parasite_load`

Grouping variable:

- `injury_status`

Number of groups:

- 2

4.1 Visualize

```
# Make injury labels easier to read
data <- data %>%
  mutate(
    injury_label = if_else(injury_status == 1, "Injured", "Not injured")
  )

# Visualize parasite load by injury status
ggplot(data, aes(x = injury_label, y = parasite_load)) +
  geom_boxplot(outlier.colour = "red") +
  labs(
    x = "Injury status",
    y = "Parasite load",
    title = "Parasite load by injury status"
  )
```


```
# Summary by group
data %>%
  group_by(injury_label) %>%
  summarise(
    n = n(),
    mean_parasites = mean(parasite_load, na.rm = TRUE),
    median_parasites = median(parasite_load, na.rm = TRUE),
    sd_parasites = sd(parasite_load, na.rm = TRUE),
    .groups = "drop"
  )
```

```
## # A tibble: 2 x 5
##   injury_label     n mean_parasites median_parasites
##   <chr>        <int>          <dbl>             <dbl>
## 1 Injured         20          101.                94
## 2 Not injured    220           44.8               41
## # i 1 more variable: sd_parasites <dbl>
```

4.2 Wilcoxon Test

```
wilcox.test(parasite_load ~ injury_label, data = data)
```

```
##
```

```
##  Wilcoxon rank sum test with continuity correction
##
## data:  parasite_load by injury_label
## W = 4353, p-value = 4.391e-13
## alternative hypothesis: true location shift is not equal to 0
```

- `W = 4353` is the Wilcoxon test statistic (analogous to the t statistic in a t-test). You don't usually interpret it biologically.
- `p = 4.391e-13` is much smaller than 0.05, providing very strong evidence that parasite loads differ between injured and uninjured gorillas.
- **Biological interpretation:** Parasite loads differed significantly between injured and uninjured gorillas (Wilcoxon rank-sum test, $p < 0.001$). Injured gorillas tended to have higher parasite loads than uninjured gorillas.

How to interpret this

The Wilcoxon test asks:

> Do the values in one group tend to be higher or lower than the values in the other group?

This is slightly different from a t-test, which focuses on the difference between group means.

For non-normal or skewed data, it is often useful to report the median as well as the mean.

The Wilcoxon test doesn't compare means like a t-test does. Instead, it compares the locations of the two distributions (based on the ranks of the observations).

So when R says: *alternative hypothesis: true location shift is not equal to 0*

it really means: *the two groups do not come from populations centered at the same location.*

Or, even more simply: *the typical values in one group are different from the typical values in the other group.*

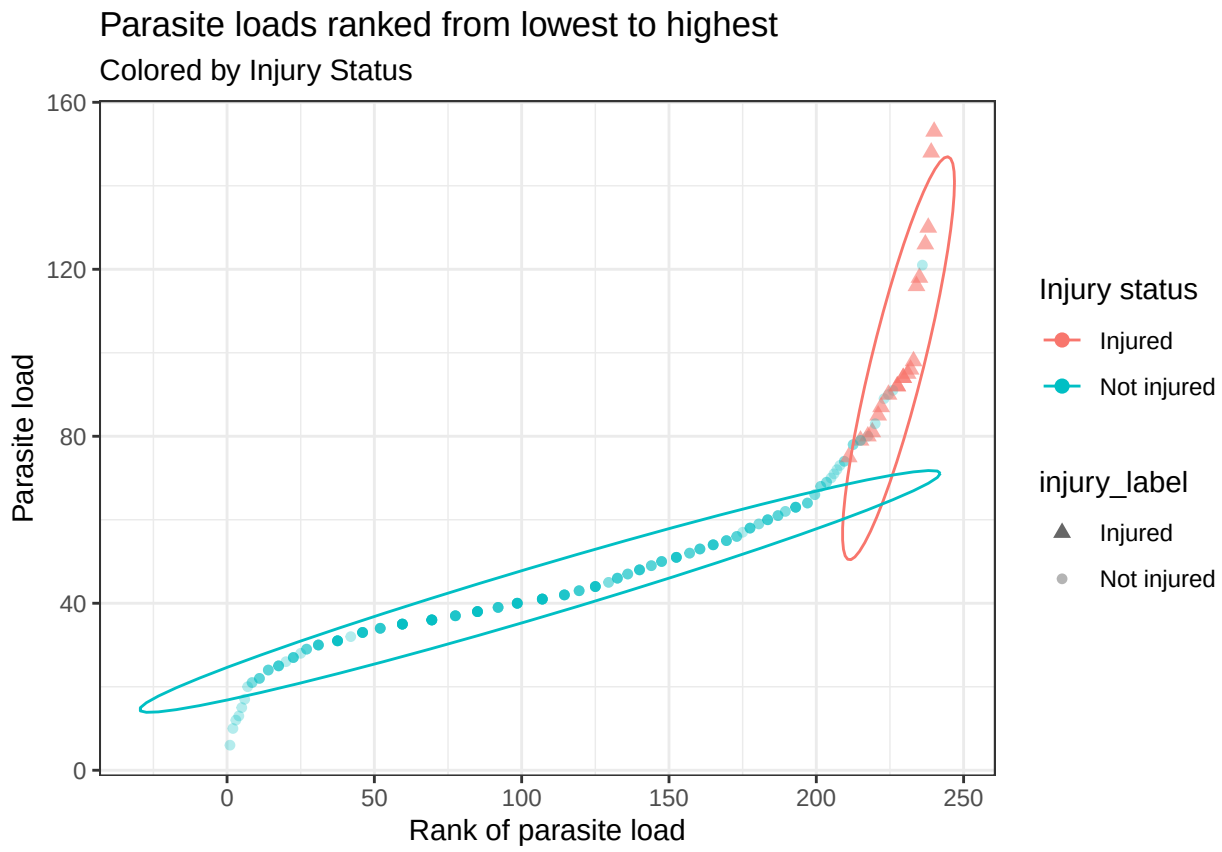
The Wilcoxon test asks whether parasite loads tend to be higher in one group than the other. It does this by ranking all of the observations from smallest to largest and comparing where the observations from each group fall in those rankings.

4.3 Ranking values

```
ranked_data <- data %>%
  mutate(
    parasite_rank = rank(parasite_load) #,
    # injury_label = if_else(injury_status == 1, "Injured", "Not injured")
  )

ggplot(ranked_data, aes(x = parasite_rank, y = parasite_load, fill = injury_label, color = injury_label
  geom_point(aes(alpha = injury_label), size = 2) +
  scale_alpha_manual(values = c(0.6, 0.3)) +
  labs(
    x = "Rank of parasite load",
    y = "Parasite load",
    color = "Injury status",
```

```
    title = "Parasite loads ranked from lowest to highest",
    subtitle = "Colored by Injury Status"
  ) +
  stat_ellipse(level = 0.95) +
  scale_shape_manual(values = c(17, 20)) +
  theme_bw()
```



The Wilcoxon test first sorts all parasite loads from the smallest to the largest and assigns each observation a rank. **If several gorillas have exactly the same parasite load, they tie**. Rather than breaking the tie arbitrarily, R gives each of them the **average of the ranks** they would have occupied

`print(sort(ranked_data$parasite_rank))` [1] 1.0 2.0 3.5 3.5 5.5 5.5 7.5 7.5 9.5 9.5 12.5 12.5...

> When ranked, observations 1 (lowest) and 2 (second lowest) each have a unique value. The 3rd and 4th observations in the dataset have the same parasite load, so either of them can be ranked 3 or ranked 4... so, they split the difference, and both are ranked 3.5

5. Kruskal-Wallis Test: Alternative to ANOVA

The Kruskal-Wallis test is often used when we want to compare three or more groups, but the response variable is not approximately normal.

Example question:

> Does parasite load differ among age classes?

Response variable:

- `parasite_load`

Grouping variable:

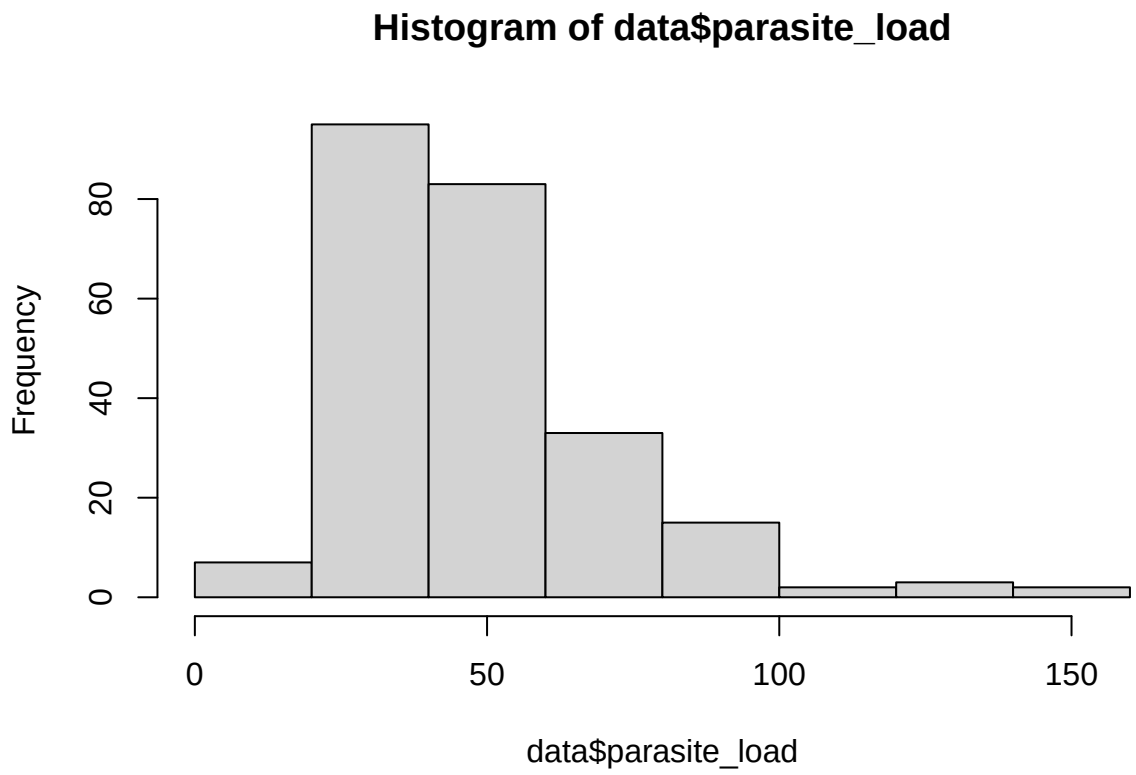
- `age_class`

Number of groups:

- 3 or more

5.1 Look at data

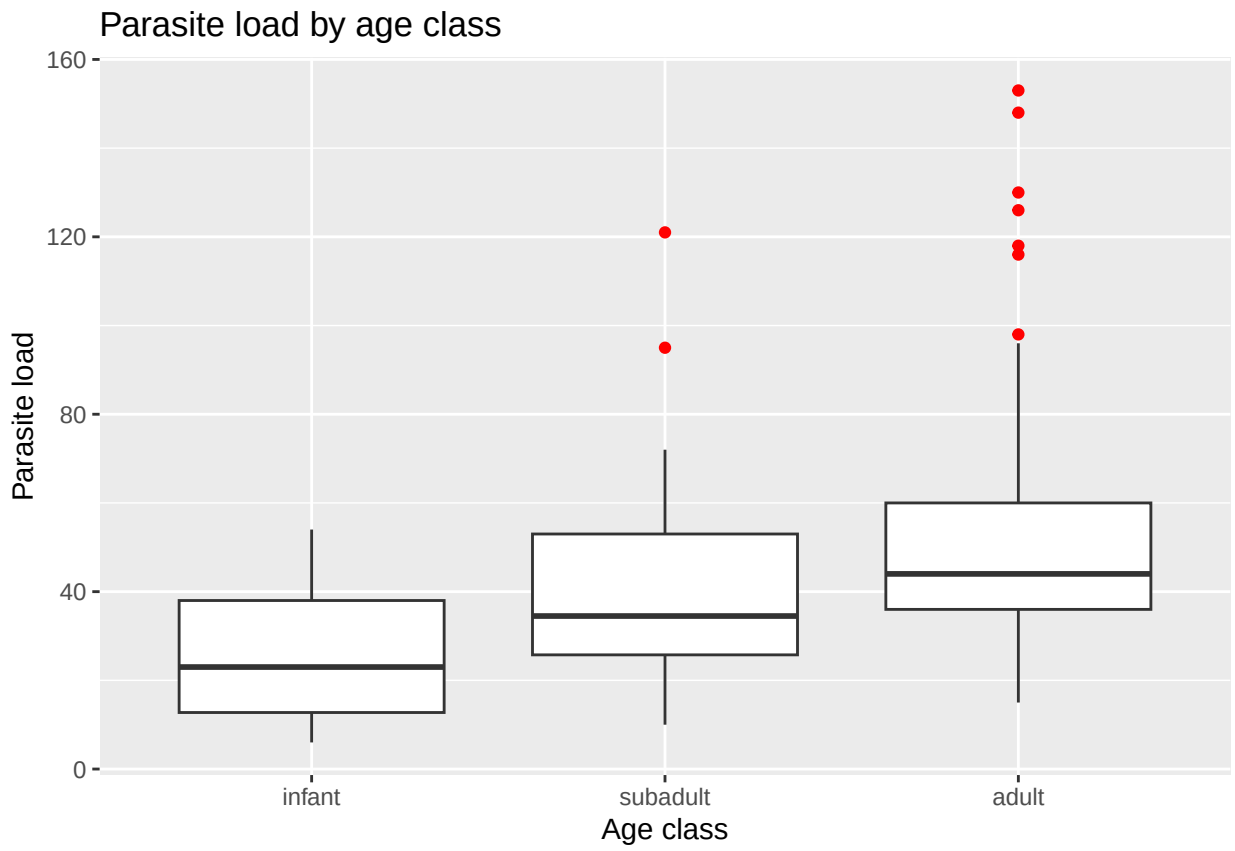
```
hist(data$parasite_load)
```




```
# Put age classes in biological order
data <- data %>%
  mutate(age_class = factor(age_class, levels = c("infant", "subadult", "adult")))

# Visualize parasite load by age class
ggplot(data, aes(x = age_class, y = parasite_load)) +
```

```
  geom_boxplot(outlier.colour = "red") +
  labs(
    x = "Age class",
    y = "Parasite load",
    title = "Parasite load by age class"
  )
```




```
# Summary by age class
summary_data <- data %>%
  group_by(age_class) %>%
  summarise(
    n = n(),
    mean_parasites = mean(parasite_load, na.rm = TRUE),
    median_parasites = median(parasite_load, na.rm = TRUE),
    sd_parasites = sd(parasite_load, na.rm = TRUE),
    .groups = "drop"
  )

summary_data
```

```
## # A tibble: 3 x 5
##   age_class     n mean_parasites median_parasites
##   <fct>     <int>          <dbl>            <dbl>
## 1 infant        8           26.1             23
## 2 subadult     16           43.6             34.5
```

```
## 3 adult       216           50.8             44
## # i 1 more variable: sd_parasites <dbl>
```

5.2 Run the Kruskal-Wallis Test

```
kruskal.test(parasite_load ~ age_class, data = data)
```

```
##
##  Kruskal-Wallis rank sum test
##
## data:  parasite_load by age_class
## Kruskal-Wallis chi-squared = 13.59, df = 2, p-value =
## 0.00112
```

The Kruskal-Wallis test asks:

> Do parasite load values tend to differ among age classes?

It does this by ranking all parasite load values from lowest to highest and then asking whether one age class tends to have consistently higher (or lower) ranks than the others.

Our results were:

`Kruskal-Wallis` X^2 `= 13.59` `Degrees of freedom = 2` (because there are three age classes) `p = 0.0011`

How to interpret this

It is similar in purpose to ANOVA, but it is used when ANOVA assumptions are not reasonable. However, again– like an ANOVA, the Kruskal-Wallis test does *not* tell us which age classes differ. It only tells us that at least one group differs from the others.

Biological interpretation: Parasite loads differed significantly among infant, subadult, and adult gorillas (Kruskal-Wallis test, $p = 0.001$). This suggests that age class is associated with differences in parasite burden. However, the test does not identify which age classes differ; additional post-hoc pairwise comparisons would be needed to determine that.

5.2 Post-hoc test

The Kruskal-Wallis test tells us whether at least one group differs, but not which groups differ.

If the Kruskal-Wallis test is significant, we can perform Dunn's test, the non-parametric equivalent of Tukey's HSD, to compare every pair of groups while adjusting for multiple comparisons.

Just like Tukey adjusted the p-values after ANOVA, Dunn's test adjusts the p-values because we're making several pairwise comparisons.

```
dunn_test(data, parasite_load ~ age_class,
  p.adjust.method = "holm"
)
```

```
## # A tibble: 3 x 9
##   .y.    group1 group2    n1    n2 statistic       p   p.adj
## * <chr>  <chr>  <chr>  <int> <int>     <dbl>   <dbl>   <dbl>
## 1 paras~ infant subad~     8    16      1.32 0.187   0.187
## 2 paras~ infant adult      8   216      3.11 0.00184 0.00553
## 3 paras~ subad~ adult     16   216      2.12 0.0336  0.0673
## # i 1 more variable: p.adj.signif <chr>
```

Biological Interpretation: Only infants and adults differed significantly in parasite load after correcting for multiple comparisons. Although subadults appeared to differ from adults, the adjusted p-value (0.067) was greater than 0.05, so we do not have strong enough evidence to conclude that they differ.

The Kruskal-Wallis test showed that parasite load differed among age classes. The Dunn post-hoc test revealed that this difference was driven by infants and adults, while the other pairwise comparisons were not statistically significant after adjusting for multiple comparisons.

Notice that the Subadult vs Adult comparison had a **raw `p-value = 0.034`**, which would normally be considered significant. However, after adjusting for multiple comparisons, the **adjusted** p-value increased to `0.067`, so we no longer consider that comparison statistically significant. This adjustment helps reduce the chance of false positives when making several comparisons.

6. Count Data

Now we will move from nonparametric group comparisons to models designed specifically for count data.

Count data are common in ecology and wildlife research.

Examples:

- number of aggression events
- number of parasites
- number of nests
- number of vocalizations
- number of offspring
- number of animal sightings

A **Poisson** distribution describes the pattern we often see when counting how many times something happens. A Poisson distribution has several characteristics:

- The values are counts (0, 1, 2, 3, . . .)
- Negative values are impossible.
- Smaller counts are usually more common than larger counts.
- The distribution is often right-skewed (many small values, few large values).
- A very important property of a Poisson distribution is that **the mean and variance are expected to be approximately equal**.

6.1 Summary

```
# Inspect count variables
summary(data$play_events)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.000   0.000   0.000   1.825   3.000  13.000
```

```r
# Mean and variance of aggression events
mean(data$play_events)
```

```
## [1] 1.825
```

```r
var(data$play_events)
```

```
## [1] 8.965063
```

> The mean tells us the average number of play events. The variance tells us how much those counts vary around that average.

Key idea

For a Poisson distribution, the mean and variance are expected to be similar.

If the variance is much larger than the mean, that may be evidence of **overdispersion**.

In our data: Mean = 1.83 Variance = 8.97

The variance is almost **more than 4 times** the mean, suggesting the data are more variable than a Poisson distribution would predict. Although the average gorilla has about 2 play events, **the individual gorillas differ from each other quite a lot**. Some have 0, some have 1–3, and a few have 10 or more. Those large differences make the variance much larger than the mean.

Consder the following hypothetical data: | Play events | Mean | Variance | | ————— | —: | ——-: | | 2, 2, 2, 2, 2 | 2 | 0 | | 1, 2, 2, 3, 2 | 2 | small | | 0, 0, 1, 4, 13 | 3.6 | large |

> The average alone doesn't tell the whole story—the variance tells us how spread out the counts are.

7. Why Not Just Use Linear Regression?

Let's ask:

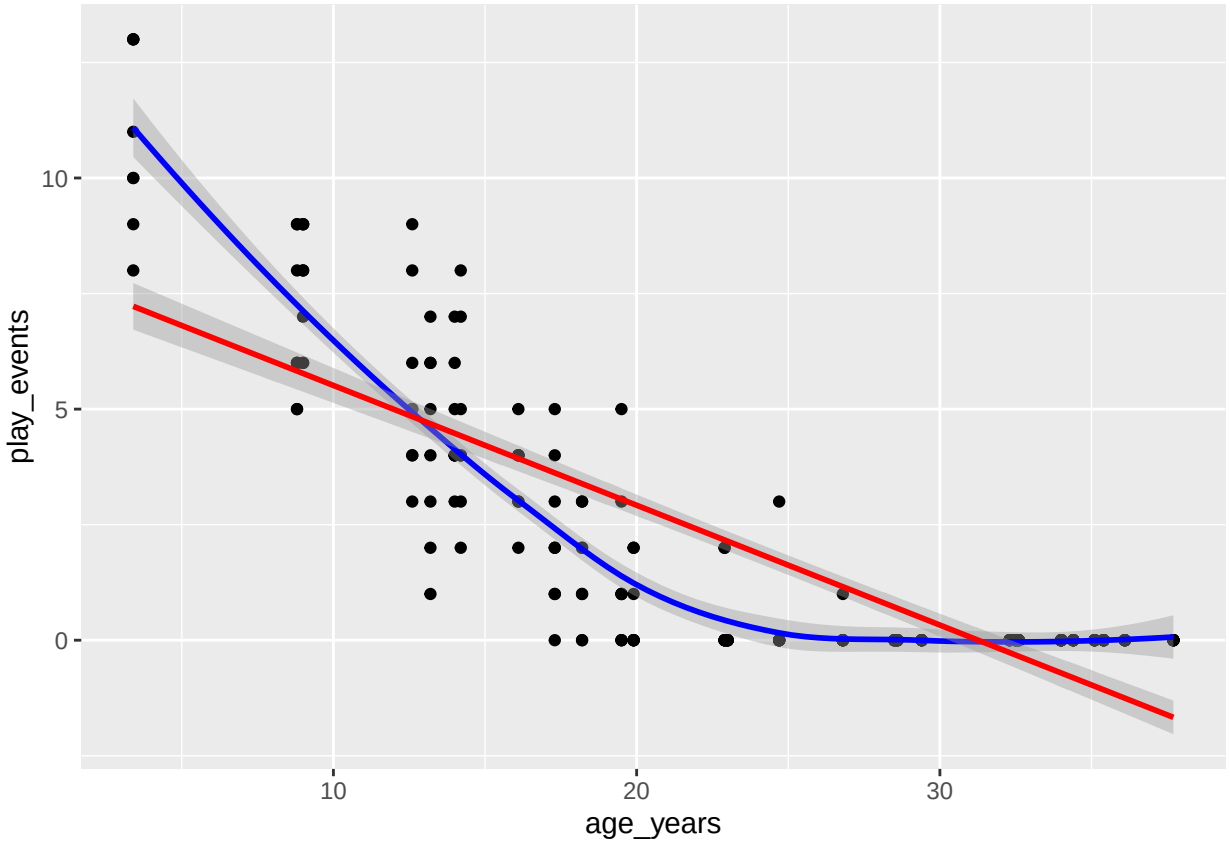
> Do older gorillas have higher parasite loads?

A simple linear model might seem like an obvious option.

7.1 Visualize

```r
ggplot(data, aes(x = age_years, y = play_events)) +
  geom_point() +
  geom_smooth(method = "loess", color = "blue") +
  geom_smooth(method = "lm", color = "red")
```

```
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
```




```
# We can fit a linear model...
mod_lm <- lm(play_events ~ age_years, data = data)

# but it will generate potentially negative predicted numbers, even though this is count data, which ca
summary(mod_lm)
```

```
##
## Call:
## lm(formula = play_events ~ age_years, data = data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.6826 -0.9458  0.2688  0.9946  5.7769
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  8.10453    0.29115   27.84   <2e-16 ***
## age_years   -0.25923    0.01117  -23.20   <2e-16 ***
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.661 on 238 degrees of freedom
## Multiple R-squared:  0.6934, Adjusted R-squared:  0.6921
## F-statistic: 538.2 on 1 and 238 DF,  p-value: < 2.2e-16
```

Interpret it:

- `Intercept (8.10)`: a newborn gorilla (age = 0) is predicted to have about 8.1 play events.
- `Slope (-0.259)`: for every additional year of age, the predicted number of play events decreases by about 0.26 events.

Instead, we use something called a Poisson regression:

8. Poisson Regression

Poisson regression is a type of generalized linear model, or GLM. It is commonly used when the response variable is a count.

Poisson regression is a type of **generalized linear model**, or **GLM**. It is commonly used when the response variable is a count. A GLM extends linear regression so that we can analyze these different types of response variables. Instead of changing what we're trying to do, a GLM changes **how the model describes the response variable**.

In R, we use: `glm(..., family = "poisson")`

Example question:

> Does play_events change with age?

Response variable:

- `play_events`, a count

Predictor variable:

- `age_years`, continuous

8.1 Generalized linear model

```
# Poisson regression
mod_pois <- glm(
  play_events ~ age_years,
  data = data,
  family = "poisson"
)

summary(mod_pois)
```

```
##
## Call:
## glm(formula = play_events ~ age_years, family = "poisson", data = data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  3.333063   0.088439   37.69   <2e-16 ***
## age_years   -0.160549   0.006681  -24.03   <2e-16 ***
```

```
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 1027.20  on 239  degrees of freedom
## Residual deviance:  200.99  on 238  degrees of freedom
## AIC: 496.41
##
## Number of Fisher Scoring iterations: 5
```

Our biological question: Do older gorillas engage in fewer play events?

We fit a Poisson regression because our response variable `play_events` is a count.

Step 1. Look at the slope `age_years` `Estimate = -0.1605` `p < 2e-16`

The negative coefficient tells us that as gorillas get older, the expected number of play events decreases.

- **Very small p-value** = the relationship is unlikely to be explained by random sampling variation alone; it's significant.
- **Deviance:** The model fits the data much better after including age than it did before age was included.
- **AIC** is mainly useful for comparing competing models. A smaller AIC generally indicates a better-fitting model.

Biological interpretation: Older gorillas tend to engage in fewer play events than younger gorillas.

More specifically: *Poisson regression showed that age was a significant predictor of play events ($p < 0.001$). The negative coefficient indicates that older gorillas engaged in fewer play events. After converting the coefficient from the log scale, we estimate that each additional year of age is associated with approximately a 15% decrease in the expected number of play events.*

Interpretation:

```
- Estimate ->          Direction and size of the relationship (on the log scale)
- p-value    ->          Is there evidence of a relationship?
exp(coef())   ->       Biological interpretation (percent change in expected count)
Residual deviance ->     The model fits much better than the null model
AIC ->                 Used only to compare competing models (e.g., Poisson vs. Negative Binomial)
```

Step 2. Now, let's convert to the original scale. Remember that Poisson regression is fitted on the log scale. To interpret the effect on the original count scale, we exponentiate the coefficient:

8.1 Interpreting Poisson Coefficients

Poisson regression uses a log link.

That means the coefficients are on the log scale. To make them easier to interpret, we exponentiate them using `exp()`.

```r
# Coefficients on the log scale
coef(mod_pois)
```

```
## (Intercept)   age_years
##   3.3330633  -0.1605488
```

```r
# > coef(mod_pois)
# (Intercept)   age_years
#   3.3330633  -0.1605488

# Exponentiated coefficients
exp(coef(mod_pois))
```

```
## (Intercept)   age_years
##  28.0240566   0.8516763
```

```r
# > exp(coef(mod_pois))
# (Intercept)   age_years
#  28.0240566   0.8516763

# This is the natural exponential function, and
# it's the inverse of the natural logarithm (log() in R).

# > exp(3.333)
# [1] 28.02228
# At age = 0, the model predicts about 28 play events.

# > exp(-0.1605)
# [1] 0.8517178
# For every additional year of age, the expected number of play events is multiplied by 0.852 (only 85.
```

> `exp()`, or `e^x` is the natural exponential function, and it's the inverse of the natural logarithm (`log()`).

So, `exp(3.333) = e^3.333 = 28.02` `exp(-0.1605) = e^-0.1605 = 0.852`

The exponentiated coefficient tells us **how the expected count changes for a one-unit increase in the predictor**. It tells us how much we multiply the expected count by for each one-unit increase in the predictor.

For example: if `exp(coef)` for age is 0.85, then each additional year of age is associated with a 15% decrease in expected play events. Year 1: 10
Year 2: 8.52
Year 3: 7.26
Year 4: 6.19... and so on

> In a **linear regression**, every additional year changes the response by **adding** or **subtracting** the same amount.
> In a **Poisson regression**, every additional year changes the response by **multiplying** by the same number.

9. Predicted Values from the Poisson Model

Predicted values help us understand the model in the original units of the response variable. Before we can draw the prediction line from our Poisson model, we need a set of ages at which we want to make predictions.

Our original dataset contains observations only at the ages we happened to sample. Instead, we'd like to predict the expected number of play events across the entire age range, from the youngest to the oldest gorilla.

9.1 Wrangle

```
# This code creates a new dataset called new_age that contains
# 100 evenly spaced ages covering the full range of our data.

# Create new data for prediction
new_age <- tibble(
  age_years = seq(
    from = min(data$age_years, na.rm = TRUE),
    to = max(data$age_years, na.rm = TRUE),
    length.out = 100
  )
) # seq() creates a sequence of numbers between the two ages, min/max.

# Now that we have a list of ages, we can ask the model:
# "For each of these ages, how many play events would you expect?"

# That's exactly what predict() does.
# It does not predict for the original dataset.
# It predicts for this new dataset containing 100 evenly spaced ages,
# based on the model we calculated.

# Get predicted values on the response scale
new_age$predicted_play_events <- predict(
  mod_pois,
  newdata = new_age,
  type = "response"
)
# Uses the fitted Poisson model (mod_pois) to make predictions.
# Remember: we're not fitting a new model here. We're using the model we already built.

# By default, Poisson regression predicts on the log scale,
# because that's how the model is fitted.
# type = "response" converts those predictions back to the original scale of the data.

head(new_age)
```

```
## # A tibble: 6 x 2
##   age_years predicted_play_events
##       <dbl>                 <dbl>
## 1      3.4                   16.2
## 2      3.75                  15.4
## 3      4.09                  14.5
```

```
## 4      4.44                  13.7
## 5      4.79                  13.0
## 6      5.13                  12.3
```

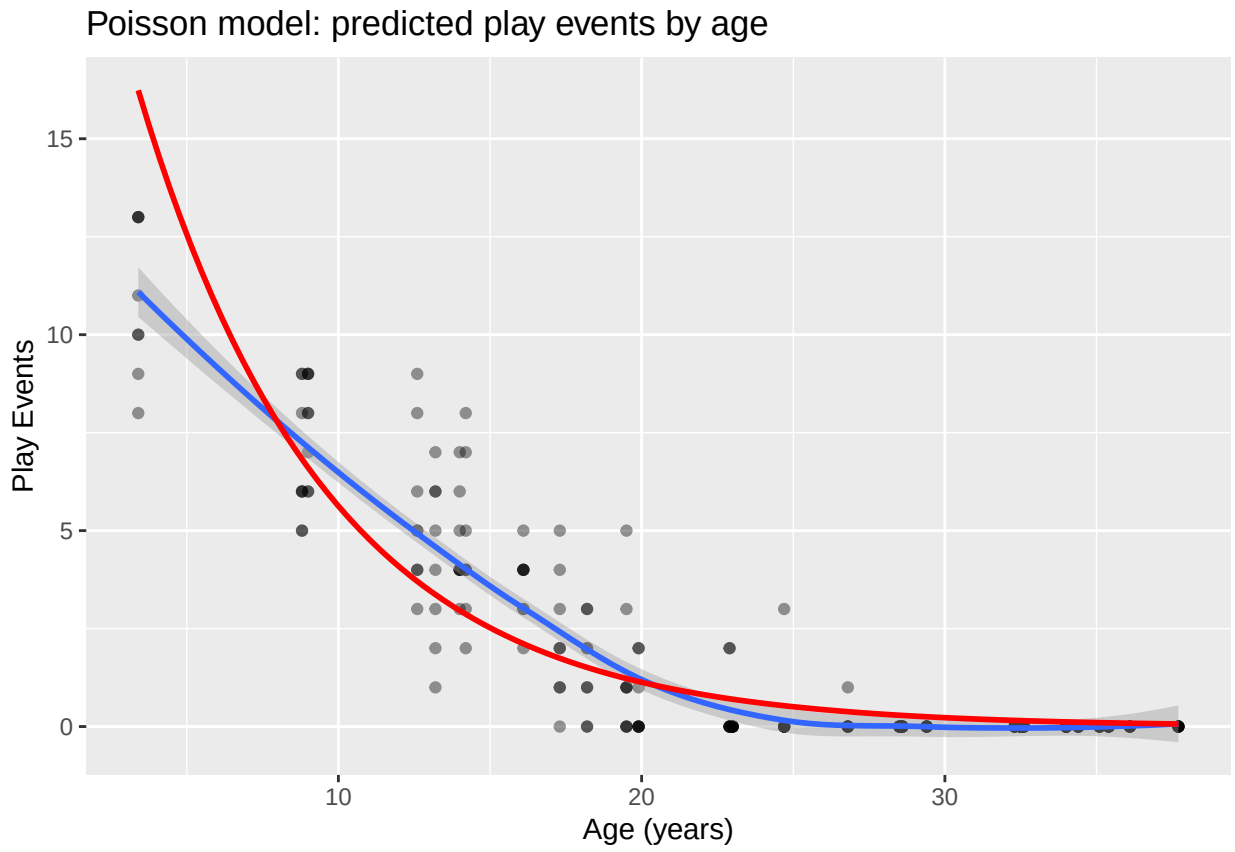
```
tail(new_age)
```

```
## # A tibble: 6 x 2
##   age_years predicted_play_events
##       <dbl>                 <dbl>
## 1      36.0                0.0870
## 2      36.3                0.0823
## 3      36.7                0.0779
## 4      37.0                0.0737
## 5      37.4                0.0697
## 6      37.7                0.0659
```

9.2 Visualize

```
# Plot observed data and predicted values
ggplot(data, aes(x = age_years, y = play_events)) +
  geom_point(alpha = 0.4) +
  geom_smooth() +
  geom_line(data = new_age, aes(x = age_years, y = predicted_play_events),
    color = "red",
    linewidth = 1
  ) +
  labs(
    x = "Age (years)",
    y = "Play Events",
    title = "Poisson model: predicted play events by age"
  )
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~
## x'
```



This plot is useful because it shows the model prediction in the original units of the data.

10. Summary: Choosing a Model When Normality Fails

Ask:

1. What is the response variable?
2. Is it quantitative, count, binary, or proportion?
3. Does the response look approximately normal?
4. Are there many zeros?
5. Is the variance much larger than the mean?
6. Are we comparing groups or making predictions?
7. Which model matches the response variable?

A simplified guide:

Situation	Possible analysis
Two groups, non-normal quantitative response	Wilcoxon test
Three or more groups, non-normal quantitative response	Kruskal-Wallis test
Count response, mean similar to variance	Poisson regression
Count response, variance much larger than mean	Negative binomial regression
Positive continuous response, right-skewed	Consider log transformation
Binary response	Logistic regression, next week

Skills Application – Lab

11. Lab Challenge: Non-normal and Count Data

For this lab, you will work with your own dataset.

Your goal is to identify whether you have a response variable that is non-normal, skewed, or count-based, and choose an appropriate analysis.

Task List

1. Load your dataset and continue working in your script.
2. Choose one response variable that is not approximately normal, or choose one count variable.
3. Inspect the response variable using `summary()` and a plot.
4. Decide what type of response variable it is:
    - skewed continuous
    - count
    - binary
    - proportion
5. If you are comparing groups:
    - consider `wilcox.test()` for two groups
    - consider `kruskal.test()` for three or more groups
6. If your response is a count:
    - fit a Poisson model with `glm(..., family = "poisson")`
    - check for overdispersion
    - if needed, fit a negative binomial model with `MASS::glm.nb()`
7. Write 2-3 sentences interpreting your results biologically.